

Foundations of DL Assignment: *Online Glyph Classification and Maths Expression Recognition*

version 0.2 - October 2025

Assignment Overview

This is a **3-part assignment** that progressively builds your skills in deep learning for handwritten Maths expression recognition. Each part uses the same stroke-based input format but with different learning objectives, architectures, and output representations. Your implementation should be in PyTorch building on `nn.Module`. Start with a baseline model first, then iterate to improve performance. Part 3 is challenging, it may take several attempts to get right. In particular, training time may be long, several days on a good GPU.

Part 1: CNN for Glyph Classification

Build a Convolutional Neural Network to classify individual handwritten digits and mathematical symbols.

- Debug and develop your model using small datasets first (e.g., `glyph_1k.h5`), then scale up.
- Use the `glyph_80k.h5` dataset for this task.
- Study the `startup_glyph_dataset.ipynb` notebook to understand the data format and preprocessing options.
- The list of glyphs includes digits 0–9 and symbols `+`, `-`, `*`, `/`, `.`, `(`, `)`, `=`.

This is a classification task, so the output (Y) is a single token per sample without `<bos>` or `<eos>` tokens. You can choose to preprocess the input strokes (X) by flattening them or keeping them as 2D coordinates. You may also implement an offline data strategy as a baseline, rendering the strokes to a bitmap and using a standard image-based CNN.

Part 2: RNN for Infix Expression Recognition

Build a Recurrent Neural Network (LSTM/GRU) to map stroke sequences to infix mathematical expressions (standard Maths notation).

- Debug and develop your model using small datasets first (e.g., `infix_1k.h5`), then scale up.
- Use the `infix_88k.h5` dataset for this task.
- Study the `startup_infix_dataset.ipynb` notebook to understand the data format and preprocessing options.
- The list of symbols includes digits 0–9 and symbols `+`, `-`, `*`, `/`, `.`, `(`, `)`, `=`.
- A typical infix expression looks like `4*(5.3+2)=`.

This is a sequence-to-sequence task, so the output (Y) is a sequence of tokens including `<bos>` and `<eos>`. You should use many-to-many RNN architectures (encoder-decoder). Watch out for variable-length sequences and padding. Also consider gradient clipping to stabilize training. Checkpoint your best models based on validation loss.

Part 3: Transformer for Postfix Expression Recognition

Build a Transformer model to convert stroke sequences to postfix notation (Reverse Polish Notation). For example, the infix expression `(3*(5.4+2))/8=` is represented in postfix as `3 <eon> 5.4 <eon> 2 <eon> + * 8 <eon> / =`.

- Debug and develop your model using small datasets first (e.g., `postfix_1k.h5`), then scale up.
- Use the `postfix_208k.h5` dataset for this task.
- Study the `startup_postfix_dataset.ipynb` notebook to understand the data format and preprocessing options.
- The list of symbols includes digits 0–9 and symbols `+`, `-`, `*`, `/`, `.`, `(`, `)`, `=`, `,` (comma used mark the end of numerals).

This is a sequence-to-sequence task, so the output (Y) is a sequence of tokens including `<bos>` and `<eos>`. You should use Transformer architectures (encoder-decoder). Although you may experiment with the transformer hyperparameters, a good starting point is to use 4 attention heads, 4 layers, and a model dimension of 128 or 256. Implement positional encoding and appropriate attention masks. Training may take several days on a good GPU, so consider using learning rate warm-up and check-pointing.

Submission

Submit your notebooks, and a brief report summarizing your approach and results for each part. Include any challenges faced and how you addressed them. Ensure your code is well-documented and organized. Add comments in your notebooks, similar to the provided tutorial notebooks.

Understanding the Data

Input Format (X)

- **2D stroke coordinates:** [sequence_length, VECTOR_SIZE] stored as `torch.float32`
- VECTOR_SIZE is 128
- Strokes are prepended with `<bos>` token ([2,...,2]) and appended with `<eos>` token ([3,...,3])
- Padded with `PAD_VALUE = -5`
- Each point in a stroke is a coordinate pair (x, y)

Output Format (Y)

- **Mathematical expression strings** tokenized to indices
- Always includes `<bos>` and `<eos>` tokens
- Padded with `PAD_VALUE = -5`

Available Datasets

```
datasets/  
├── glyph_1k.h5          # 1K samples - single digit/symbol classification  
├── glyph_80k.h5         # 80K samples - single digit/symbol classification  
├── infix_1k.h5          # 1K samples - infix notation (e.g., "3+5*2")  
├── infix_88k.h5         # 88K samples - infix notation  
├── postfix_1k.h5        # 1K samples - postfix notation (e.g., "3,5,2*,+")  
└── postfix_208k.h5      # 208K samples - postfix notation
```

Part 1: CNN for Glyph Classification

Objective

Build a **Convolutional Neural Network (CNN)** to classify individual handwritten digits and mathematical symbols.

What you'll learn: Image classification, CNNs, handling stroke-based input data

Dataset: `glyph_80k.h5`

- **Input (X):** Stroke sequences for single glyphs
- **Output (Y):** Single token (digit 0-9 or symbol +, -, *, /, ., (,), =)
- **Vocab size:** 18 tokens (no special tokens needed)

Getting Started

1. **Open:** `startup_glyph_dataset.ipynb`
2. **Explore:** Run the notebook cells to understand the data format
3. **Key differences from Parts 2 & 3:**
 - Y does NOT include `<bos>` or `<eos>` tokens
 - This is a classification task, not sequence-to-sequence
 - Vocab is simpler (no special tokens)

Data Processing Strategy

```
# Glyph preprocessing options:
preprocess = {'x': True, 'y': False} # Flatten x, keep y as single token
# OR
preprocess = {'x': False, 'y': False} # Keep x as list of strokes, y as single token
```

With x preprocessing ('x': True):

- Removes <bos>, <eos>, and padding strokes
- Flattens coordinates: $[x^{1_1}, y^{1_1}, \dots, x^{1_{n1}}, y^{1_{n1}}, -1, x^{2_1}, y^{2_2}, \dots, x^{2_{n2}}, y^{2_{n2}}, -1, \dots]$
- -1 marks stroke segment boundaries
- Good for 1D CNN or fully connected networks

Without x preprocessing ('x': False):

- Keeps stroke structure: $[[x^{1_1}, y^{1_1}, \dots, x^{1_{n1}}, \text{padding}], [x^{2_1}, y^{2_1}, \dots, x^{2_{n2}}, \text{padding}], \dots]$
- Good for 2D CNN treating strokes as image-like data

Model Architecture Hints

- **Input layer:** Match your chosen preprocessing (1D flattened or 2D strokes)
- **CNN layers:** Extract spatial/sequential features from strokes
- **Output layer:** 18 classes (one per token)
- **Loss function:** CrossEntropyLoss

Vocabulary

```
{0: '0', 1: '1', 2: '2', 3: '3', 4: '4', 5: '5', 6: '6', 7: '7', 8: '8', 9: '9', 10: '+', 11: '-',
12: '*', 13: '/', 14: '.', 15: '(', 16: ')', 17: '='}
```

NO special tokens (<bos>, <eos>, <unk>, <pad>)

Expected Output Shape

- **X batch:** [batch_size, seq_len] if flattened OR [batch_size, num_strokes, 2] if 2D
- **Y batch:** [batch_size] - single token per sample

Part 2: RNN for Infix Expression Recognition

Assignment Objective

Build a **Recurrent Neural Network (RNN/LSTM/GRU)** to map stroke sequences to infix mathematical expressions (standard math notation).

What you'll learn: Sequence-to-sequence learning, RNNs/LSTMs, handling variable-length sequences, teacher forcing

Dataset: infix_88k.h5

- **Input (X):** Stroke sequences for full expressions
- **Output (Y):** Infix notation token sequences (e.g., $3+5*2=$ $\rightarrow$ ['<bos>', '3', '+', '5', '*', '2', '=', '<eos>'])
- **Vocab size:** 23 tokens (19 base + 4 special tokens)

Getting Started

1. **Open:** startup_infix_dataset.ipynb
2. **Explore:** Run the notebook to see sequence-to-sequence data
3. **Key differences from Part 1:**
 - Y includes <bos> and <eos> tokens (sequence-to-sequence task)

- Output is a **sequence of tokens**, not a single classification
- Variable-length sequences require padding and masking
- Vocab includes special tokens: `<unk>` , `<pad>` , `<bos>` , `<eos>`

Data Processing Strategy

```
# Recommended preprocessing for RNN:
preprocess = {'x': True, 'y': True} # Flatten x with -1 separators, add <bos>/<eos> to y
```

Model Architecture Hints

- **Encoder:** RNN/LSTM/GRU to process input stroke sequences
- **Decoder:** RNN/LSTM/GRU to generate output token sequence autoregressively
- **Attention** (optional): Helps model focus on relevant parts of input strokes
- **Input:** Flattened stroke sequences (1D) or embedded stroke features
- **Output:** Sequence of token indices from vocab (with `<bos>` and `<eos>`)
- **Loss function:** CrossEntropyLoss (use `ignore_index=PAD_VALUE` to skip padding in loss)

Vocabulary

```
{0: '<unk>', 1: '<pad>', 2: '<bos>', 3: '<eos>', 4: '0', 5: '1', 6: '2', 7: '3', 8: '4', 9: '5',
10: '6', 11: '7', 12: '8', 13: '9', 14: '+', 15: '-', 16: '*', 17: '/', 18: '.', 19: '(', 20: ')',
21: '='}
```

Training Tips

- Use teacher forcing during training
- Handle variable-length sequences with padding
- Mask loss computation for padded tokens

Part 3: Transformer for Postfix Expression Recognition

Assignment Objective

Build a **Transformer model** (encoder-decoder) to map stroke sequences to postfix notation (Reverse Polish Notation).

What you'll learn: Transformer architecture, self-attention, positional encoding, autoregressive generation

Dataset: `postfix_208k.h5`

- **Input (X):** Stroke sequences for full expressions
- **Output (Y):** Postfix notation token sequences (e.g., "3+52=" → `['3', '+', '5', '2', '=', '']`)
- **Vocab size:** 23 tokens (19 base + 4 special tokens)
- **Note:** `,` (comma) represents "end of number" token `<eon>` in postfix notation

Getting Started

1. **Open:** `startup_postfix_dataset.ipynb`
2. **Explore:** Run the notebook to understand postfix notation format
3. **Key differences from Parts 1 & 2:**
 - Largest dataset (208K samples)
 - Postfix notation: operands before operators
 - Comma separates numbers

Data Processing Strategy

```
# Recommended preprocessing for Transformer:
preprocess = {'x': False, 'y': True} # Keep x as array of strokes, add <bos>/<eos> to y
# OR
preprocess = {'x': True, 'y': True} # Flatten x for simpler processing
```

Model Architecture Hints

- **Encoder:** Transformer encoder to process input stroke sequences. Treat strokes as token with positional encoding
- **Decoder:** Transformer decoder for autoregressive token generation
- **Multi-head Self-Attention:** Critical for capturing dependencies between strokes and tokens
- **Positional Encoding:** Essential for capturing stroke order and token position
- **Attention Masks:**
 - Causal masking for decoder (prevent looking ahead)
 - Padding masks for both encoder and decoder
- **Loss function:** CrossEntropyLoss (use `ignore_index=PAD_VALUE` to ignore padding)

Vocabulary

```
{0: '<unk>', 1: '<pad>', 2: '<bos>', 3: '<eos>', 4: '0', 5: '1', 6: '2', 7: '3', 8: '4', 9: '5',
10: '6', 11: '7', 12: '8', 13: '9', 14: '+', 15: '-', 16: '*', 17: '/', 18: '.', 19: '(', 20: ')',
21: '=', 22: ',','}'
```

Training Tips

- Use the largest possible batch size
- Implement learning rate warm-up
- Monitor attention patterns for interpretability

Common Data Pipeline (All Parts)

All three parts of the assignment share this data loading structure:

```
# 1. Setup paths
import os, sys
from pathlib import Path
b_paths = [os.path.abspath('.'), os.path.abspath(os.path.join('.', 'datasets'))]
for b_path in b_paths:
    if b_path not in sys.path:
        sys.path.append(b_path)

# 2. Define H5Dataset with preprocessing options
class H5Dataset(Dataset):
    def __init__(self, file_path, vocab_obj, split='train', preprocess={'x': False, 'y': True}):
        # ... see startup notebooks for full implementation

# 3. Create vocabulary (task-specific)
vocab_obj = vocab(ordered_dict, specials=[...]) # Glyph: no specials; Infix/Postfix: add specials

# 4. Load datasets
train_data = H5Dataset(H5_FILE, vocab_obj, split='train', preprocess={'x': ?, 'y': ?})
valid_data = H5Dataset(H5_FILE, vocab_obj, split='valid', preprocess={'x': ?, 'y': ?})
test_data = H5Dataset(H5_FILE, vocab_obj, split='test', preprocess={'x': ?, 'y': ?})

# 5. Create DataLoaders with custom collate function
train_loader = DataLoader(train_data, batch_size=128, shuffle=True, collate_fn=pad_collate)
```

Visualization & Debugging

All startup notebooks include SVG visualization:

```
svg_str = strokes_to_svg(x_item, {'height':100, 'width':100}, -5, 2, 3)
display(SVG(data=svg_str))
```

This renders strokes as SVG paths to visually inspect the handwriting.

Decoding Predictions

```
decoded = ''
for i in y_item:
    if i != PAD_VALUE:
        token = vocab_obj.lookup_token(int(i))
        token = token.replace('*', 'x') # multiply symbol
        token = token.replace('/', '÷') # divide symbol
        token = token.replace(',', '<eon>') # end of number
        decoded = decoded + token
```

Quick Reference

Constants Across All Parts

- **PAD_VALUE**: Always -5 for both X and Y padding
- **Batch Size**: Standard is 128 (adjust based on GPU memory)

Special Token Indices (Parts 2 & 3 only)

- <unk> : 0 (unknown token)
- <pad> : 1 (padding token)
- <bos> : 2 (beginning of sequence)
- <eos> : 3 (end of sequence)

Dataset Sizes

Part	Dataset File	Training Samples	Task Type
1	glyph_80k.h5	80,000	Classification
2	infix_88k.h5	88,000	Seq2Seq
3	postfix_208k.h5	208,000	Seq2Seq

Vocabulary Sizes

- **Part 1 (Glyph)**: 18 tokens (no special tokens)
- **Parts 2 (Infix)**: 23 tokens (18 base + 4 special)
- **Parts 3 (Postfix)**: 23 tokens (19 base + 4 special)

Notation Differences

- **Infix**: Standard maths notation (e.g., 3+5.4*2)
- **Postfix**: Reverse Polish Notation (e.g., 3, 5.4, 2, * +)
 - Operands come before operators
 - Comma separates numbers

Development Workflow

Recommended Approach for Each Part:

Step 1: Understand the Data

- Run the appropriate `startup_*.ipynb` notebook for your current part
- Examine batch shapes and preprocessing options
- Visualize samples using SVG rendering to see actual handwriting

Step 2: Design Your Model

- **Part 1:** CNN architecture for classification (18 output classes)
- **Part 2:** Encoder-decoder RNN/LSTM for sequence-to-sequence (vocab size 22)
- **Part 3:** Transformer encoder-decoder for sequence-to-sequence (vocab size 23)
- Consider preprocessing strategy based on your architecture

Step 3: Implement Training Loop

- Define model, optimizer, loss function
- Handle padding in loss computation (`ignore_index=PAD_VALUE` for Parts 2 & 3)
- Track validation performance
- Save best model checkpoints

Step 4: Evaluate & Iterate

- Test on held-out test set
 - Visualize predictions vs ground truth
 - Calculate accuracy metrics (classification accuracy for Part 1, sequence accuracy for Parts 2 & 3 e.g. character error rate, edit distance...)
 - Analyse failure cases and iterate
-

Assignment Progression

Start with Part 1 (CNN/Glyph)

- Simplest task: single symbol classification
- Build fundamentals of working with stroke data
- Learn CNN architecture and training
- Succeed with high accuracy (>90%)

Progress to Part 2 (RNN/Infix)

- Learn sequence-to-sequence modeling
- Understand encoder-decoder architecture
- Master handling variable-length sequences and teacher forcing

Complete with Part 3 (Transformer/Postfix)

- Apply Transformer architecture
- Understand self-attention mechanisms
- Learn to work with larger datasets (208K samples)
- This part is challenging, expect to spend significant time debugging and training